

torch.median#

<https://pytorch.org/docs/stable/generated/torch.median.html>

Description:

Returns the median of the values in input. The median is not unique for input tensors with an even number of elements. In this case the lower of the two medians is returned. To compute the mean of both medians, use `torch.quantile()` with `q=0.5` instead. This function produces deterministic (sub)gradients unlike `median(dim=0)`

Function Signature

`torch.median(input) → Tensor#`

Parameters

`torch.median(input, dim=-1, keepdim=False, *, out=None)`

Parameters

Keyword Arguments

Parameters

Keyword

out ((Tensor, Tensor), optional) – The first tensor will be populated with the median values and the second tensor, which must have dtype long, with their indices in the dimension dim of input.

Code Examples

```
>>> a = torch.randn(1, 3)
>>> a
tensor([[ 1.5219, -1.5212,  0.2202]])
>>> torch.median(a)
tensor(0.2202)
```

```
>>> a = torch.randn(4, 5)
>>> a
tensor([[ 0.2505, -0.3982, -0.9948,  0.3518, -1.3131],
        [ 0.3180, -0.6993,  1.0436,  0.0438,  0.2270],
        [-0.2751,  0.7303,  0.2192,  0.3321,  0.2488],
        [ 1.0778, -1.9510,  0.7048,  0.4742, -0.7125]])
>>> torch.median(a, 1)
torch.return_types.median(values=tensor([-0.3982,  0.2270,
0.2488,  0.4742]), indices=tensor([1, 4, 4, 3]))
```

Notes & Warnings

NOTE

Note The median is not unique for input tensors with an even number of elements. In this case the lower of the two medians is returned. To compute the mean of both medians, use `torch.quantile()` with `q=0.5` instead.

⚠ WARNING

Warning This function produces deterministic (sub)gradients unlike `median(dim=0)`

NOTE

Note The median is not unique for input tensors with an even number of elements in the dimension `dim`. In this case the lower of the two medians is returned. To compute the mean of both medians in input, use `torch.quantile()` with `q=0.5` instead.

⚠ WARNING

Warning indices does not necessarily contain the first occurrence of each median value found, unless it is unique. The exact implementation details are device-specific. Do not expect the same result when run on CPU and GPU in general. For the same reason do not expect the gradients to be deterministic.

Detailed Documentation

Documentation

Returns the median of the values in input.

The median is not unique for input tensors with an even number of elements. In this case the lower of the two medians is returned. To compute the mean of both medians, use `torch.quantile()` with `q=0.5` instead.

This function produces deterministic (sub)gradients unlike `median(dim=0)`

`input (Tensor)` – the input tensor.

Returns a namedtuple (values, indices) where values contains the median of each row of input in the dimension `dim`, and indices contains the index of the median values found in the dimension `dim`.

By default, `dim` is the last dimension of the input tensor.

If `keepdim` is `True`, the output tensors are of the same size as input except in the dimension `dim` where they are of size 1. Otherwise, `dim` is squeezed (see `torch.squeeze()`), resulting in the outputs tensor having 1 fewer dimension than input.

The median is not unique for input tensors with an even number of elements in the dimension `dim`. In this case the lower of the two medians is returned. To compute the mean of both medians in input, use `torch.quantile()` with `q=0.5` instead.

`indices` does not necessarily contain the first occurrence of each median value found, unless it is unique. The exact implementation details are device-specific. Do not expect the same result when run on CPU and GPU in general. For the same reason do not expect the gradients to be deterministic.

`input (Tensor)` – the input tensor.

`dim` (int, optional) – the dimension to reduce. If `None`, all dimensions are reduced.

`keepdim` (bool, optional) – whether the output tensor has `dim` retained or not. Default: `False`.

`out` ((Tensor, Tensor), optional) – The first tensor will be populated with the median values and the second tensor, which must have dtype `long`, with their indices in the dimension `dim` of input.